



Structuri de Date și Algoritmi (SDA)

As.Drd.Ing. Bianca Gușiță

AC/TI, AN II

LAB 10-11 - Liste. Structuri derivate din liste



Laborator 10-11 - Liste

$$L = a_1, a_2, \dots, a_n \quad (n \geq 0)$$

O listă L este o secvență de zero sau mai multe elemente, numite noduri, toate fiind de același tip de bază T .

Dacă $n \geq 1$, a_1 se spune că este primul nod al listei, iar a_n , ultimul nod. Dacă $n = 0$, lista este vidă.

O proprietate importantă a unei liste este aceea că nodurile sale pot fi ordonate liniar în funcție de poziția lor în cadrul listei.

Se spune că a_i precede pe a_{i+1} ($i = 1, 2, \dots, n-1$), iar a_i succede pe a_{i-1} ($i = 2, 3, \dots, n$), a_i aflându-se pe poziția i .

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului tablou



O listă se asimilează cu un tablou, nodurile fiind elementele tabloului memorate în locații succesive de memorie.

Traversarea unei liste

În această reprezentare, lista poate fi ușor traversată folosind indecșii tabloului.

Insertia nodurilor

Noile noduri pot fi inserate la sfârșitul listei, cu eficiența $O(1)$. Insertia unui nod în interiorul listei presupune deplasarea tuturor nodurilor următoare cu o poziție spre sfârșitul listei, astfel rezultă o complexitate $O(n)$.

Suprimarea nodurilor

În cazul suprimării, aceasta presupune deplasarea tuturor nodurilor următoare cu o poziție spre începutul listei.

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului tablou



Tipul Lista se definește ca un articol cu două câmpuri:

-primul este *un tablou cu elementele de tip Nod*, cu lungimea aleasă astfel încât să poată acoperi cea mai mare dimensiune de lista ce apare în respectiva aplicație;

-al doilea câmp reprezintă *poziția ultimului nod al listei*

Cod

```
#define LungMax 100
```

```
typedef struct {
```

```
    Nod tablou[LungMax];
```

```
    int ultim;
```

```
} Lista;
```

Laborator 10-11 – Liste. Caz particular - Stiva



Stiva poate fi văzută ca un caz particular de listă cu proprietatea ca **ultimul element inserat este primul element extras.**

Exemplu de stiva implementata cu tablou:

```
#define nmax 100
• typedef struct{
• char corp_stiva[nmax];
• int varf_stiva;
• } stiva; // o stiva de dimensiune maxima nmax
• stiva st; //variabila de tip stiva
• void push(char car) //pune in stiva un caracter

• {
• st.corp_stiva[st.varf_stiva]=car; //se adauga un caracter in varf
• st.varf_stiva++; //se marcheaza adaugarea acestuia
• }
• char pop() //ia din stiva un caracter
• {
• st.varf_stiva--;
• return st.corp_stiva[st.varf_stiva]; //se ia din varf un caracter
• }
```

Laborator 10-11 – Liste. Caz particular - Coada



Coada poate fi văzută ca un caz particular de listă cu proprietatea ca **primul element inserat este primul element extras**.

Reprezentarea listelor cu ajutorul tablourilor, prezentată la începutul acestui subcapitol, poate fi utilizată și pentru cozi, dar nu este foarte eficientă.

Indicatorul la ultimul element al listei, permite implementarea simplă, într-un număr fix de pași $O(1)$, a operației de adăugare a unui element.

Scoaterea unui element presupune însă *mutarea întregii cozi cu o poziție în tablou*, operație care necesită un timp $O(n)$, dacă coada are lungimea n .

Pentru a depăși acest dezavantaj se poate utiliza o structură de tablou circular în care prima poziție urmează ultimei.

Laborator 10-11 – Liste. Caz particular – Coada



Exemplu de
implementare
coadă circulară
cu ajutorul
tablourilor:

```
1  #define nmax 30 //in practica dimensiunea maxima utila este nmax-1
2  typedef struct coada
3  {
4      char elemente[nmax]; //elemente de tip char
5      int citire;
6      int scriere;
7  }coada;
8
9  coada init(coada c) //initializeaza pointerii de citire si de scriere pt
10     // coada data
11  {
12      c.citire = 0;
13      c.scriere = nmax - 1;
14      return c;
15  }
16  int vid(coada c) //verifica daca coada este vida
17  {
18      if (c.citire == (c.scriere + 1) % nmax) return 1;
19      return 0;
20  }
21  int plina(coada c)
22  {
23      if (c.citire == (c.scriere + 2) % nmax)
24          //verificare daca coada este plina
25          //(daca scriere este cu 2 pozitii in urma pointerului de citire)
26      return 1;
27      else return 0;
28  }
```

Laborator 10-11 – Liste. Caz particular - Coada



Exemplu de
implementare
coadă circulară
cu ajutorul
tablourilor:

```
30 coada adauga(coada c, char car)
31 {
32     if (!plina(c))
33     {
34         c.scriere = (c.scriere + 1) % nmax;
35         //se avanseaza cu capatul de scriere
36         c.elemente[c.scriere] = car; //se adauga elementul
37     }
38     return c;
39 }
40
41 char scoate(coada *c) //se scoate un element din coada circulara
42 {
43     char element = '\0';
44     if (!vid(*c))
45     {
46         element = c->elemente[c->citire];
47         c->citire = ((c->citire) + 1) % nmax;
48     }
49     return element;
50 }
```


Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer



Implementarea listelor cu ajutorul tipului pointer. Structuri recursive de tip listă

Cu ajutorul tipului pointer, se definește structura unui nod al listei liniare, având pe lângă informațiile utile și cel puțin un câmp de legătură către alt nod.

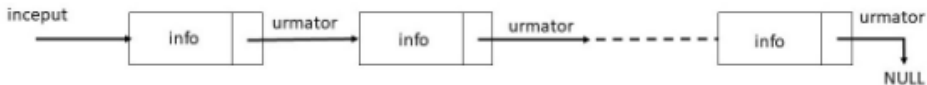
Dacă avem un singur câmp de legătură, lista se numește simplu înlănțuită.

Câmpul cheie servește la identificarea nodului, câmpul următor e pointer de înlănțuire la nodul următor, iar cel info conține informația utilă.

Variabila început indică spre primul nod al listei; în unele situații în locul lui început se utilizează un nod fictiv, adică o variabilă de tip nod cu câmpurile cheie și info neprecizate, dar câmpul următor indicând spre primul nod al listei.

De asemenea uneori este util a se păstra pointerul spre ultimul nod al listei.

O variantă de listă înlănțuită este lista circulară la care dispăre noțiunea de prim, ultim nod, lista fiind un pointer ce se plimbă pe lista.



Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă simplu înlănțuită



În C, o listă simplu înlănțuită poate fi implementată folosind pointeri și structuri. Fiecare nod al listei conține două elemente: o valoare și un pointer către următorul nod.

1. Definirea structurii nodului

Folosim o structură Node pentru a reprezenta fiecare nod din listă.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct Node {
5      int data;
6      struct Node* next;
7  } Node;
```

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă simplu înlănțuită



2. Crearea unui nod nou

Funcția createNode alocă memorie pentru un nod nou și îl inițializează.

```
Node* createNode(int data) {  
    Node* newNode = (Node*)malloc(sizeof(Node));  
    newNode->data = data;  
    newNode->next = NULL;  
    return newNode;  
}
```

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă simplu înlănțuită



3. Adăugarea unui nod la începutul listei

Funcția `insertAtBeginning` adaugă un nod nou la începutul listei.

```
void insertAtBeginning(Node** head, int data) {  
    Node* newNode = createNode(data);  
    newNode->next = *head;  
    *head = newNode;  
}
```

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă simplu înlănțuită



4. Adăugarea unui nod la sfârșitul listei

Funcția insertAtEnd adaugă un nod la sfârșitul listei.

```
void insertAtEnd(Node** head, int data) {
    Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}
```

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă simplu înlănțuită



5. Afișarea listei

Funcția printList parcurge lista și afișează fiecare element.

```
void printList(Node* head) {  
    Node* temp = head;  
    while (temp != NULL) {  
        printf("%d -> ", temp->data);  
        temp = temp->next;  
    }  
    printf("NULL\n");  
}
```

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă simplu înlănțuită



6. Ștergerea unui nod

Funcția deleteNode șterge primul nod cu o valoare specificată.

```
void deleteNode(Node** head, int key) {
    Node* temp = *head;
    Node* prev = NULL;

    if (temp != NULL && temp->data == key) {
        *head = temp->next;
        free(temp);
        return;
    }

    while (temp != NULL && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL) return;

    prev->next = temp->next;
    free(temp);
}
```

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă simplu înlănțuită



7. Eliberarea memoriei

Funcția freeList eliberează memoria tuturor nodurilor.

```
void freeList(Node* head) {  
    Node* temp;  
    while (head != NULL) {  
        temp = head;  
        head = head->next;  
        free(temp);  
    }  
}
```


Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă simplu înlănțuită



8. Main

```
int main() {
    Node* head = NULL;

    insertAtBeginning(&head, 3);
    insertAtBeginning(&head, 1);
    insertAtEnd(&head, 5);
    insertAtEnd(&head, 7);

    printf("Lista inițială: ");
    printList(head);

    deleteNode(&head, 5);
    printf("Lista după ștergerea nodului 5: ");
    printList(head);

    freeList(head);
    return 0;
}
```

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă dublu înlănțuită



O listă **dublu înlănțuită** este similară cu o listă simplu înlănțuită, dar fiecare nod are doi pointeri: unul către nodul următor și unul către nodul anterior. Asta permite parcurgerea listei în ambele direcții (înainte și înapoi).

1. Definirea structurii nodului

Folosim o structură Node care conține trei elemente: datele (valoarea nodului), un pointer către nodul următor (next) și un pointer către nodul anterior (prev).

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
} Node;
```

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă dublu înlănțuită



2. Crearea unui nod nou

Funcția createNode alocă memorie pentru un nod nou și îl inițializează.

```
Node* createNode(int data) {  
    Node* newNode = (Node*)malloc(sizeof(Node));  
    newNode->data = data;  
    newNode->next = NULL;  
    newNode->prev = NULL;  
    return newNode;  
}
```

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă dublu înlănțuită



3. Adăugarea unui nod la începutul listei

Funcția insertAtBeginning adaugă un nod nou la începutul listei.

```
void insertAtBeginning(Node** head, int data) {  
    Node* newNode = createNode(data);  
    newNode->next = *head;  
    if (*head != NULL) {  
        (*head)->prev = newNode;  
    }  
    *head = newNode;  
}
```

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă dublu înlănțuită



4. Adăugarea unui nod la sfârșitul listei

Funcția insertAtEnd adaugă un nod la sfârșitul listei.

```
void insertAtEnd(Node** head, int data) {
    Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;
}
```

Laborator 10-11 - Liste. Implementarea listelor cu ajutorul tipului pointer. Listă dublu înlănțuită



5. Afișarea listei de la început la sfârșit

Funcția printListForward parcurge lista de la început la sfârșit și afișează fiecare element.

```
void printListForward(Node* head) {  
    Node* temp = head;  
    while (temp != NULL) {  
        printf("%d -> ", temp->data);  
        temp = temp->next;  
    }  
    printf("NULL\n");  
}
```

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă dublu înlănțuită



6. Afișarea listei de la sfârșit la început

Funcția printListBackward parcurge lista de la sfârșit la început, oferind o afișare inversată.

```
void printListBackward(Node* head) {
    if (head == NULL) return;

    Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->prev;
    }
    printf("NULL\n");
}
```

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă dublu înlănțuită



7. Ștergerea unui nod

Funcția deleteNode șterge primul nod care conține o valoare specificată.

```
void deleteNode(Node** head, int key) {
    Node* temp = *head;

    while (temp != NULL && temp->data != key) {
        temp = temp->next;
    }

    if (temp == NULL) return; // Nodul nu a fost găsit

    if (temp->prev != NULL) {
        temp->prev->next = temp->next;
    } else {
        *head = temp->next; // Ștergem capul listei
    }

    if (temp->next != NULL) {
        temp->next->prev = temp->prev;
    }

    free(temp);
}
```


Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă dublu înlănțuită



8. Eliberarea memoriei

Funcția freeList eliberează memoria tuturor nodurilor.

```
void freeList(Node* head) {  
    Node* temp;  
    while (head != NULL) {  
        temp = head;  
        head = head->next;  
        free(temp);  
    }  
}
```

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă dublu înlănțuită



9. Main

```
int main() {
    Node* head = NULL;

    insertAtBeginning(&head, 3);
    insertAtBeginning(&head, 1);
    insertAtEnd(&head, 5);
    insertAtEnd(&head, 7);

    printf("Lista de la început la sfârșit: ");
    printListForward(head);

    printf("Lista de la sfârșit la început: ");
    printListBackward(head);

    deleteNode(&head, 5);
    printf("Lista după ștergerea nodului 5: ");
    printListForward(head);

    freeList(head);
    return 0;
}
```

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă simplu înlănțuită VS Lista dublu înlănțuită



Asemănări între listele simplu și dublu înlănțuite

Structură bazată pe noduri: Ambele sunt alcătuite din noduri care conțin o valoare și un pointer (sau pointeri) pentru a indica legăturile între ele.

Acces secvențial: Ambele liste permit parcurgerea elementelor în mod secvențial, de la un nod la altul.

Dimensiune variabilă: Spre deosebire de tablouri (arrays), listele înlănțuite sunt structuri dinamice, putând crește sau micșora în dimensiune în funcție de cerințele programului.

Inserare și ștergere de elemente: Atât listele simplu, cât și cele dublu înlănțuite permit adăugarea și eliminarea nodurilor în diferite poziții (la început, sfârșit, sau la o anumită poziție) fără a muta alte elemente din memorie.

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă simplu înlănțuită VS Lista dublu înlănțuită



Deosebiri între listele simplu și dublu înlănțuite

Numărul de pointeri per nod:

1. **Lista simplu înlănțuită:** Fiecare nod conține un pointer către nodul următor.
2. **Lista dublu înlănțuită:** Fiecare nod conține doi pointeri – unul către nodul următor și unul către nodul anterior.

Direcția de parcurgere:

1. **Lista simplu înlănțuită:** Permite parcurgerea doar într-o singură direcție (de la început spre sfârșit).
2. **Lista dublu înlănțuită:** Permite parcurgerea în ambele direcții (de la început spre sfârșit și invers).

Complexitatea manipulării nodurilor:

1. **Lista simplu înlănțuită:** Inserarea și ștergerea unui nod în interiorul listei poate fi puțin mai rapidă deoarece nu necesită ajustarea pointerului pentru nodul anterior.
2. **Lista dublu înlănțuită:** Inserarea și ștergerea unui nod în interiorul listei necesită modificarea ambelor pointere (next și prev), ceea ce poate necesita mai multe operații.

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă simplu înlănțuită VS Lista dublu înlănțuită



Deosebiri între listele simplu și dublu înlănțuite

Memorie:

- 1.Lista simplu înlănțuită:** Fiecare nod are un singur pointer, deci ocupă mai puțină memorie pe nod.
- 2.Lista dublu înlănțuită:** Fiecare nod are doi pointeri, deci ocupă mai multă memorie, ceea ce poate fi relevant în aplicații cu cerințe stricte de memorie.

Flexibilitatea accesului:

- 1.Lista simplu înlănțuită:** Datorită faptului că se poate parcurge doar într-o singură direcție, nu se poate reveni ușor la nodul anterior.
- 2.Lista dublu înlănțuită:** Nodurile pot fi accesate mai flexibil, fiind posibil să se navigheze și înainte, și înapoi, ceea ce poate simplifica anumite operații (de exemplu, găsirea și ștergerea unui element după un criteriu anume).

Utilizare în practică:

- 1.Lista simplu înlănțuită:** Este preferată când e necesar doar un acces secvențial și simplu la date, economisind memorie.
- 2.Lista dublu înlănțuită:** Este mai potrivită pentru aplicații în care este necesară o navigare bidirecțională, cum ar fi implementările de structuri de date complexe (ex. dequeues, lista de istoric a unui browser).

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă simplu înlănțuită. Avantaje și Dezavantaje



Lista simplu înlănțuită

Avantaje

Consum redus de memorie:
Fiecare nod are un singur pointer (next), ceea ce face lista mai economică din punct de vedere al memoriei comparativ cu lista dublu înlănțuită.

Implementare mai simplă:
Structura și manipularea unei liste simplu înlănțuite sunt mai ușor de implementat, cu mai puține operații pentru a lega și delga nodurile.

Performanță mai bună pentru unele operații:
Inserarea și ștergerea nodurilor poate fi mai rapidă, pentru că necesită actualizarea doar a unui singur pointer (next).

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă simplu înlănțuită. Avantaje și Dezavantaje



**Lista simplu
înlănțuită**

Dezavantaje

Acces unidirecțional:
Parcurgerea este posibilă doar într-o singură direcție, de la început spre sfârșit, ceea ce poate fi limitativ pentru anumite aplicații.

Dificultate în operațiile care cer acces bidirecțional:
Pentru a șterge un nod intermediar sau pentru a reveni la nodul anterior, lista simplu înlănțuită poate necesita parcurgerea întregii liste de la capăt.

Mai puțin flexibilă: Orice operație de căutare sau inserare într-o poziție specifică trebuie să parcurgă lista de la început, ceea ce poate fi ineficient pentru liste mari.

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă dublu înlănțuită. Avantaje și Dezavantaje



Lista dublu
înlanțuită

Avantaje

**Acces
bidirecțional:**
Poate fi parcursă
în ambele direcții,
ceea ce face
operațiile de
navigare mai
flexibile și
eficiente.

**Ștergere și
inserare mai
simplă:** Pentru
ștergerea unui nod
intermediar sau
pentru anumite
operații de
inserare, accesul
la nodul anterior
face aceste
operații mai rapide
și mai ușor de
implementat.

**Flexibilitate
sporită:** Ideală
pentru structuri de
date care necesită
navigare în ambele
direcții (de
exemplu, în cazul
unei liste de
navigare, cum ar fi
istoria unui
browser).

Laborator 10-11 – Liste. Implementarea listelor cu ajutorul tipului pointer. Listă dublu înlănțuită. Avantaje și Dezavantaje



Lista simplu înlănțuită

Dezavantaje

Consum crescut de memorie: Fiecare nod are doi pointeri (next și prev), ceea ce duce la un consum de memorie mai mare.

Complexitate crescută: Codul pentru inserarea și ștergerea nodurilor este mai complicat decât în cazul listei simplu înlănțuite, deoarece trebuie actualizați doi pointeri pentru fiecare operație.

Posibilitate mai mare de erori: Datorită complexității mai mari, este mai ușor să apară erori în implementare, de exemplu, să se creeze referințe încrucișate greșite, ceea ce poate duce la bucle infinite sau erori de accesare a memoriei.



Laborator 10-11 – Liste circulare

O listă circulară este o variantă de listă înlanțuită (poate fi simplu sau dublu înlanțuită) în care ultimul nod al listei nu mai indică NULL, ci se conectează înapoi la primul nod, formând astfel un cerc. Această structură permite parcurgerea listei într-un ciclu continuu, fără a avea nevoie de o limită explicită de final.

Tipuri de liste circulare

- 1. Lista simplu circulară:** Este o listă simplu înlanțuită în care ultimul nod are un pointer către primul nod, iar primul nod are un pointer doar către nodul următor.
- 2. Lista dublu circulară:** Este o listă dublu înlanțuită în care ultimul nod are un pointer către primul nod, iar primul nod are un pointer înapoi către ultimul nod, permițând navigarea în ambele direcții într-un ciclu.



Laborator 10-11 – Liste circulare

Avantajele listelor circulare

Parcursere ciclică: Se poate parcurge lista în mod repetat de la început la sfârșit, fără a ajunge la capătul listei (nu există NULL la sfârșit), ceea ce poate fi util pentru aplicații care necesită o buclă continuă (de exemplu, cicluri de procesare sau jocuri).

Mai eficientă pentru anumite operații: Accesul la începutul listei după ce s-a ajuns la final este mai rapid, deoarece nu este nevoie să se revină la începutul listei – se continuă pur și simplu de unde s-a ajuns.

Utilă pentru implementarea de structuri circulare: Listele circulare sunt utilizate adesea în structuri de date care necesită acces continuu, cum ar fi cozile circulare, algoritmi de partajare echitabilă a resurselor (de exemplu, round-robin în sistemele de operare), sau pentru navigare circulară într-o interfață grafică.



Laborator 10-11 – Liste circulare

Dezavantajele listelor circulare

Complexitate sporită: Codul pentru parcurgere și manipulare poate fi mai dificil de gestionat, deoarece trebuie să se aibă grijă să nu se creeze bucle infinite. Este necesar un punct de referință de început pentru a ști unde se oprește parcurgerea.

Acces și gestionare mai complicate: Deși lista permite parcurgerea ciclică, accesul la elementele de la început sau de la sfârșit poate fi mai puțin intuitiv, deoarece nu există un „capăt” clar definit.



Laborator 10-11 – Liste circulare

**Exemple de
utilizare**

Listele
circulare sunt
frecvent
folosite în:

**Cozile
circulare**
pentru buffer-e
circulare, unde
după
adăugarea
ultimului
element, se
revine la
primul.

**Algoritmul
round-robin** în
managementul
proceselor din
sisteme de
operare.

**Structuri
circulare** în
jocuri și
interfețe
grafice, cum ar
fi listele de
selectare care
revin la primul
element după
ultimul.



Laborator 10-11 – Liste – Multiliste

O **multilistă** este o structură de date complexă care permite stocarea de informații legate între ele prin mai multe criterii, permițând astfel accesul și organizarea datelor din mai multe perspective.

Într-o multilistă, fiecare nod poate avea mai mulți pointeri, astfel încât același element să aparțină mai multor liste simultan. Aceasta este o extensie a listei înlănțuite, unde un nod poate face parte din mai multe lanțuri de noduri.



Laborator 10-11 – Liste – Multiliste

Exemplu de Multilistă în C

Vom crea un exemplu simplu de multilistă în care fiecare nod este conectat într-o listă de tip "rând" și într-o listă de tip "coloană".

Structura nodului:
Nodul va avea trei câmpuri: data (datele nodului), un pointer nextRow pentru a face legătura cu următorul nod din rând și un pointer nextCol pentru a face legătura cu următorul nod din coloană.

Funcții pentru gestionarea nodurilor: Vom include funcții pentru crearea unui nod, adăugarea nodurilor în rânduri și coloane, și afișarea datelor din structură.

Laborator 10-11 – Liste - Multiliste

```
#include <stdio.h>
#include <stdlib.h>

// Structura nodului pentru multilistă
typedef struct Node {
    int data;
    struct Node* nextRow; // Legătura către următorul nod din rând
    struct Node* nextCol; // Legătura către următorul nod din coloană
} Node;

// Funcție pentru crearea unui nod nou
Node* createNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->nextRow = NULL;
    newNode->nextCol = NULL;
    return newNode;
}

// Funcție pentru adăugarea unui nod într-un rând
void addNodeToRow(Node** rowHead, int data) {
    Node* newNode = createNode(data);
    if (*rowHead == NULL) {
        *rowHead = newNode;
    } else {
        Node* temp = *rowHead;
        while (temp->nextRow != NULL) {
            temp = temp->nextRow;
        }
        temp->nextRow = newNode;
    }
}
```


Laborator 10-11 - Liste - Multiliste

```
// Funcție pentru adăugarea unui nod într-o coloană
void addNodeToColumn(Node** colHead, int data) {
    Node* newNode = createNode(data);
    if (*colHead == NULL) {
        *colHead = newNode;
    } else {
        Node* temp = *colHead;
        while (temp->nextCol != NULL) {
            temp = temp->nextCol;
        }
        temp->nextCol = newNode;
    }
}

// Funcție pentru afișarea datelor pe rânduri
void printRows(Node* rowHead) {
    Node* rowTemp = rowHead;
    while (rowTemp != NULL) {
        Node* temp = rowTemp;
        while (temp != NULL) {
            printf("%d -> ", temp->data);
            temp = temp->nextRow;
        }
        printf("NULL\n");
        rowTemp = rowTemp->nextCol;
    }
}
```



Laborator 10-11 - Liste - Multiliste

```
// Funcție pentru afișarea datelor pe coloane
void printColumns(Node* colHead) {
    Node* colTemp = colHead;
    while (colTemp != NULL) {
        Node* temp = colTemp;
        while (temp != NULL) {
            printf("%d -> ", temp->data);
            temp = temp->nextCol;
        }
        printf("NULL\n");
        colTemp = colTemp->nextRow;
    }
}

// Program principal pentru testarea multilistei
int main() {
    Node* rowHead = NULL;
    Node* colHead = NULL;

    // Adăugăm noduri în rând și coloană
    addNodeToRow(&rowHead, 1);
    addNodeToRow(&rowHead, 2);
    addNodeToRow(&rowHead, 3);

    addNodeToColumn(&colHead, 4);
    addNodeToColumn(&colHead, 5);
    addNodeToColumn(&colHead, 6);

    printf("Afișarea datelor pe rânduri:\n");
    printRows(rowHead);

    printf("\nAfișarea datelor pe coloane:\n");
    printColumns(colHead);

    // Eliberăm memoria
    // Aceasta este un exemplu simplu, dar ar trebui adăugată o funcție pentru eliberarea memoriei pentru fiecare nod.

    return 0;
}
```

Laborator 10-11 - Liste - Multiliste



Avantaje:

- Flexibilitate mare în organizarea datelor: o multilistă permite accesarea nodurilor în funcție de mai multe criterii.
- Parcurgere eficientă pentru structuri complexe: potrivită pentru grafice, tabele și structuri multidimensionale.

Dezavantaje:

- Complexitate crescută: necesită un management mai atent al memoriei și al pointerilor.
- Consum de memorie mai mare: fiecare nod trebuie să stocheze mai mulți pointeri.